Machine Learning Documentation

Iris Decision Tree Classification Report

Technical architecture, training evaluation, and decision boundaries analysis.

TEST ACCURACY	TOTAL SAMPLES	MAX TREE DEPTH	TARGET CLASSES
96.67%	150	3	3

1. Executive Summary & Scope

This technical document provides an evaluation of the Decision Tree Classifier built on the classic Iris dataset. The workflow encompasses data ingestion, exploratory health checks, stratified split partitioning, hyperparameter constrained model fitting, and performance benchmarking. The model reaches high predictive precision while maintaining readability with a constrained maximum depth of 3.

2. Model Performance Benchmarks

Evaluated on holdout test dataset (n = 30, stratified across all species):

SPECIES / TARGET CLASS	PRECISION	RECALL	F1-SCORE	SUPPORT
Iris-setosa	1.00	1.00	**1.00**	10
Iris-versicolor	1.00	0.90	**0.95**	10
Iris-virginica	0.91	1.00	**0.95**	10
Macro / Weighted Average	**0.97**	**0.97**	**0.97**	**30**

Key Technical Finding

The separation of *Iris-setosa* is linearly deterministic based on `petal_length ≤ 2.45 cm`. Secondary differentiation between *versicolor* and *virginica* relies primarily on `petal_width` thresholds.

3. Data Pipeline & Training Strategy

Data Preprocessing

- Zero missing values detected across 150 rows.
- Balanced classes: 50 samples per flower species.
- Stratified split: 80% Train (120) / 20% Test (30).

Hyperparameter Setup

- Criterion: `Gini Impurity`
- Max Depth: 3 (regularized against overfitting)
- Random State: 42 (reproducible results)

4. Decision Tree Rule Architecture

Topological rule hierarchy derived during model fit:



5. Execution Code Snippet

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Split with Stratification
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Initialize and Train
clf = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
clf.fit(X_train, y_train)

# Evaluation
y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred))
```